
Phizura

On-the-Fly Music Recording with Method Proxies

Domenico Cipriani, Sebastian Jordan Montano, Stephane Ducasse

Univ. Lille, Inria, CNRS, Centrale Lille - CRIStAL

IWST 2026, Plovdiv, Bulgaria

What is live coding?

- A performance practice: artists write and modify code in real time
- The output is live - music, visuals, or poetry generated on the fly
- An improvised art form: the code is the score, written in front of the audience
- Established field - the International Conference on Live Coding (ICLC) is in its 10th year
- Popular tools: Tidal Cycles, Sonic Pi, Orca, Mercury

Coypu: live music in Pharo

- A library and DSL for on-the-fly music programming, inside the Pharo environment
- First tool to bring live coding into the Smalltalk world
- Code is written and evaluated in a Playground
- Drives an audio server - internal (Phausto DSP) or external via OSC / MIDI (e.g. SuperCollider)
 - Every evaluation hits the running sound immediately
 - The performance is what gets evaluated, when, and in what order

The problem: capturing the performance

- Audio recording captures the sound - but throws away the process
 - Large files, hard to share, impossible to re-run differently
- Compositional data captures the edits, evaluations, and their timing
 - Exact replay, with the option of different sounds
 - Lightweight text, easy to share and analyse
- The Tidal Cycles community asked for exactly this - no integrated tool existed

What Phizura sets out to do

- Record the actual writing and evaluation of expressions, not just the audio
- Capture code executions AND keystrokes, with precise timestamps
- Replay the session as self-contained, executable Pharo code
- Faithful original timing - the script is still live, modifiable, remixable

First tool to record live coding via reflective instrumentation, at evaluation granularity

First attempt: the Announcer-based recorder

The earlier approach (ESUG 2024)

- Built on Pharo's Announcer / Announcement classes - the observer pattern
- The Coypu codebase was modified: an announcement was fired at each musical event
- A recorder subscribed to those announcements and logged each with a timestamp
- It worked - but with significant trade-offs...

Why the Announcer approach hurt

- Recording logic was scattered throughout the Coypu source
- Every new recordable event meant editing Coypu: define an announcement, fire it
- Tight coupling - recorder depended on Coypu's announcement classes
- Keystrokes, cursor, other tools - would need ever more invasive changes
- Any change to the evaluation pipeline risked silently breaking the hooks

Motivated a redesign that keeps instrumentation entirely outside Coypu

Method Proxies: the enabling tool

- A per-method instrumentation library for Pharo - 'message-passing control'
- Run user code before, after, or instead of a method - and during stack unwind
- Leaves the instrumented source code completely intact
- Simple API: subclass a handler (MpHandler), override a few hooks
 - Each method is swapped for a precompiled trap - still JIT-compiled, fast
 - Meta-safe, unwind-safe, dynamic install/uninstall, ~1.5x overhead

Announcer vs Method Proxies

- Coypu changes: Announcer needed edits at every site - Method Proxies needs none
- Coupling: tight shared types - vs fully external instrumentation
- New events: edit Coypu + define announcement - vs just intercept a method
- Removal: revert Coypu edits - vs simply uninstall the proxies
- Keystrokes: not supported before - now supported
- Code emission: none before - now two modes (Performance and MoofLOD)

Phizura: architecture and implementation

Three packages, three jobs

- Phizura - the recorder, the instrumentor, and the code emitter
- Phizura-Playground - a custom Playground hosting the session and its toolbar
- Phizura-Tests - the test suite

Three responsibilities: instrument the Playground, collect events, expose the workflow

Instrumentation

- PhizuraEvaluationInstrumentor picks the methods and installs an MpMethodProxy on each
- The two methods invoked during code evaluation are intercepted
- Covers all evaluation endpoints - every evaluation is captured
- Both proxies share one handler instance
- No Coypu class is modified, subclassed, or even mentioned

The handler - the only user code needed

```
PhizuraHandler >> beforeExecutionWithReceiver:  
    anObject arguments: anArrayOfArguments  
    | argument |  
    argument := anArrayOfArguments first.  
    recorder addRecord:  
        (PhizuraEvaluationRecordEntry new  
            time: DateAndTime now;  
            expression: argument;  
            cursorPosition: playgroundPage text cursorPosition;  
            yourself)
```

The first argument is the source string about to be evaluated. Timestamp it, read the cursor, push to the recorder.

Recording the timeline

- Two kinds of entries kept in one chronological timeline:
 - PhizuraEvaluationRecordEntry - one per evaluated expression (via Method Proxies)
 - PhizuraKeyEventRecordEntry - one per keystroke (captured independently)
- PhizuraRecorder owns the entries, holds the instrumentor, toggles the recording flag
 - Start: create instrumentor, install proxies
 - Stop: uninstall proxies, clear the flag - image back to original

The Phizura Playground

- The standard Playground has no stable method to hook - text flows through Spec layers
- A thin custom subclass exposes `doEvaluateAndGoFor:`, which receives the text directly
- Adds five toolbar commands:
 - Phizurear (toggle recording), Clean Phizura (reset), Inspect Phizura
 - Open MoofLOD and Open Records - emit the two replay-script flavours

Code emission and replay

- Session ends as a time-ordered list of timestamped entries
- PhizuraCodeEmitter walks them, subtracts consecutive timestamps to recover delays
- Emits Pharo source separated by 'seconds wait' statements
- Wrapped in a forked block - waits don't freeze the UI, replay runs from the same Playground
- Double dispatch lets you add new formats or new entry types independently

Mode 1 - emitPerformanceCode

```
[  
  ep := EcoPhausto new.  
  1.0 seconds wait.  
  ep playFor: 128 bars.  
  1.5 seconds wait.  
  #(1 0 0 1 0 0 0 0) asSeq to: #kick.  
  1.8 seconds wait.  
  #rumba asRhythm to: #conga.  
] fork.
```

Keystrokes filtered out, only evaluations remain. Self-contained: any image with the same Coypu setup reproduces it verbatim.

Mode 2 - emitMoofLODCode

```
[  
  '1' toLocal: 'keyPressed'.  
  0.269059 seconds wait.  
  '6' toLocal: 'keyPressed'.  
  ...  
  16 plena to: #kick.  
  '16 plena to: #kick.' toLocal: 'evaluatedCommand'.  
] fork.
```

Same music, plus OSC-compatible signals for every expression, keystroke and cursor move - feeds visualisations, second screens, sync.

Related work

- Tidal Cycles forum: stated the requirements, concluded with no tool existing
- Live Writing (Lee & Essl, 2015): records keystroke-level typing in a browser via editor hooks
 - Phizura instead uses reflective instrumentation - no editor or library changes
- SHARP: visualises the version history of code blocks, but no timing or replay
 - Complementary - SHARP keeps structure, Phizura keeps the temporal performance

Generality beyond music

- Instrumentation targets ALL Playground evaluations, not Coypu-specific code
- Visual live coding (Bloc, Roassal) could be recorded with no change to Phizura
- A general-purpose prototyping recorder - capture an exploration with order and pace
- Pedagogy: an instructor's session becomes a tiny text file students replay, pause, and edit

Unlike video, the replay is live code; unlike a listing, it keeps the rhythm of thought

Limitations and future work

- Records evaluated expressions only - not un-evaluated edits, typing, or deletions
- Does not capture mouse actions (select, cut, paste via mouse)
- A single session at a time, held in a global variable
- Future: a public API for user-defined capture handlers on their own methods
- Future: recording across multiple Playgrounds for collaborative sessions

Conclusion

- Phizura is a non-intrusive recorder for live coding in Pharo
- Method Proxies instrument the Playground's evaluation methods without touching Coypu
- Timestamped expressions are emitted as executable code that replays with faithful timing
- More modular, decoupled and extensible than the Announcer approach

Method Proxies, built for profiling and debugging, work just as well for creative coding

Thank you - Questions?
